Azure IP Whitelisting Script

Technical Documentation & Line-by-Line Explanation

`Add-AzureIPRules.ps1`

Covers: Script Purpose · Parameters · Helper Functions · Resource Blocks · Scenario Testing

1. Script Overview

Add-AzureIPRules.ps1 is an Azure DevOps pipeline PowerShell script that manages IP firewall / access restriction rules across five Azure resource types in a single run. It is idempotent — running it twice with the same inputs produces the same result without duplicating rules. Each resource block is independently toggled via boolean flags, so a single pipeline can be used for any combination of resources.

> **Supported Resources:** Storage Account | Key Vault | Azure SQL Server | Function App | App Service (Web App)

Key design decisions:

- Single script, five resources — one pipeline variable group controls everything.
- Functions do NOT inherit script-scope variables in PowerShell. All values are passed explicitly as parameters to avoid the character-indexing bug (e.g. 'siva2' → 's').
- [string[]] explicit cast on every Split-CSV result ensures a single-element list is always an array, never a bare string whose [0] index returns the first character.
- Function Apps and App Services both use Get-AzWebApp — they share the same Azure resource type. Get-AzFunctionApp is unreliable across Az module versions.
- Priority conflicts are resolved automatically by walking up from the requested number until a free slot is found, and the taken-set is updated per rule within the same batch.

2. Input Parameters

All parameters are strings. Boolean toggles accept the literal string **"True"** (case-insensitive comparison). Every parameter defaults to a single space which is trimmed to empty on line 20–29.

Parameter	Type / Default	Description
`$AddStorage`	string / "false"	Set to True to run the Storage Account block
`$AddKeyVault`	string / "false"	Set to True to run the Key Vault block
`$AddSQL`	string / "false"	Set to True to run the SQL Server block
`$AddFunction`	string / "false"	Set to True to run the Function App block
`$AddAppService`	string / "false"	Set to True to run the App Service block
`$StorageAccountName`	string / " "	Exact name of the Storage Account
`$KeyVaultName`	string / " "	Exact name of the Key Vault
`$SQLServerName`	string / " "	Exact server name (not FQDN) of the SQL Server
`$FunctionAppName`	string / " "	Exact name of the Function App
`$AppServiceName`	string / " "	Exact name of the App Service / Web App
`$IPAddresses`	string / " "	Comma-separated IPs or CIDRs (/24-/31). Used by Storage, KeyVault, Function, AppService
`$RuleNames`	string / " "	Comma-separated rule names. Used by SQL, Function, AppService
`$StartIPs`	string / " "	Comma-separated start IPs for SQL firewall rules
`$EndIPs`	string / " "	Comma-separated end IPs for SQL firewall rules
`$Priorities`	string / " "	Comma-separated priority integers for Function App and App Service rules

3. Line-by-Line Explanation

Lines 1–17

```
param(

[string]$AddStorage = "false",

[string]$IPAddresses = " ",

... (16 parameters total)

)
```

Declares all 16 input parameters as [string] type. Boolean toggles (AddStorage, AddKeyVault, AddSQL, AddFunction, AddAppService) default to 'false'. Resource name and value parameters default to a single space so they are never null — they get trimmed to empty in the next block.

Lines 19–29

```
$StorageAccountName = $StorageAccountName.Trim()

$IPAddresses = $IPAddresses.Trim()

... (all 10 value params trimmed)
```

Trims leading/trailing whitespace from every value parameter. Azure DevOps pipeline variable groups sometimes inject spaces around values. Without trimming, a name like ' myapp ' would fail the resource lookup even though the resource exists.

Lines 31–40

```
$runStorage = $AddStorage -eq 'True'

$runKeyVault = $AddKeyVault -eq 'True'

...

if (-not ($runStorage -or $runKeyVault -or ...)) { exit 1 }
```

Converts the string toggle parameters into proper boolean variables. The guard on line 38 fails the pipeline immediately with a clear error if no resource is ticked — prevents a silent no-op run that wastes pipeline minutes.

Lines 47–51

```
function Split-CSV([string]$value) {

if ([string]::IsNullOrWhiteSpace($value)) { return [string[]]@() }

[string[]]$result = $value.Split([char]',') | ForEach-Object { $_.Trim() }

| Where-Object { $_ -ne '' }

return $result

}
```

CRITICAL FIX: Splits a comma-separated string into a guaranteed [string[]] array. Using PowerShell's -split operator treats the delimiter as regex and can misbehave. More importantly, without the explicit [string[]] cast, a single-element result like 'siva2' is returned as a plain [string]. Indexing a plain string with [0] returns the first CHARACTER ('s'), not the whole string — this was the root cause of the Processing: s | 1/32 | Priority 49 bug seen in pipeline logs.

Lines 53–61

```
function Resolve-IPEntry([string]$ip) {

if ($ip -match '/') {

# Validates CIDR prefix is between /24 and /31 (Azure limit)

if (...prefix -ge 24 -and prefix -le 31) { return $ip }

Write-Error 'Invalid CIDR...'; exit 1

}

if ($ip -match '^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$') { return $ip }

Write-Error 'Invalid IP'; exit 1

}
```

Validates each IP entry. If a CIDR suffix is present, it checks that the prefix length is between /24 and /31 — Azure Storage and Key Vault reject anything outside this range. Plain IPs are validated against a dotted-quad regex. The function exits the pipeline immediately on invalid input rather than sending a bad value to Azure.

Lines 63–73

```
function Resolve-FreePriority([int]$requested,

[System.Collections.Generic.HashSet[int]]$takenPriorities) {

$candidate = $requested

while ($takenPriorities.Contains($candidate)) {

$candidate++

}

return $candidate

}
```

Walks up from the requested priority number until a free slot is found. Uses a HashSet for O(1) contains checks rather than iterating an array each time. The HashSet is passed by reference so the caller can .Add() newly assigned priorities, preventing two rules in the same batch from both landing on the same auto-assigned slot.

Lines 75–82

```
function Get-WebAppRG([string]$appName) {

$app = Get-AzWebApp -Name $appName -ErrorAction SilentlyContinue

if (-not $app) {

$app = Get-AzWebApp | Where-Object { $_.Name -eq $appName }

| Select-Object -First 1

}

return $app.ResourceGroup

}
```

Resolves the Resource Group for both Function Apps and App Services using Get-AzWebApp. Function Apps ARE Web Apps in Azure (they share the same resource type, differentiated only by the 'kind' property). Get-AzFunctionApp is not available in all Az module versions and was returning null, causing pipeline failures. The fallback subscription-wide search handles cases where the service principal has restricted scope.

Lines 94–112

```
if ($runStorage) {

$resolvedIPs = Split-CSV $IPAddresses | ForEach-Object { Resolve-IPEntry $_ }

$rg = (Get-AzStorageAccount | Where-Object {

$_.StorageAccountName -eq $StorageAccountName }).ResourceGroupName

$existing = (Get-AzStorageAccount -RG $rg -Name ...).NetworkRuleSet.IpRules

foreach ($ip in $resolvedIPs) {

if ($existing -contains $ip) { [SKIP] }

else { Add-AzStorageAccountNetworkRule ... }

}

}
```

Storage Account block. Splits and validates IPs, resolves the resource group by scanning all storage accounts in the subscription, then reads the current NetworkRuleSet. For each IP, if it already exists the rule is skipped; otherwise it is added. No priority is needed for Storage — Azure manages ordering internally.

Lines 117–136

```
if ($runKeyVault) {

$existingNorm = ...IpAddressRanges | ForEach-Object { $_ -replace '/32$', '' }

foreach ($ip in $resolvedIPs) {

$ipForCheck = $ip -replace '/32$', ''

if ($existingNorm -contains $ipForCheck) { [SKIP] }

else { Add-AzKeyVaultNetworkRule ... }

}

}
```

Key Vault block. Azure stores plain IPs internally with a /32 suffix (e.g. 10.0.0.1 becomes 10.0.0.1/32). Without normalisation, the duplicate check would always fail and the same IP would be added repeatedly. Both the existing list and the candidate IP have /32 stripped before comparison.

Lines 141–181

```
  if ($runSQL) {

  $ruleList = Split-CSV $RuleNames

  $startList = Split-CSV $StartIPs

  $endList = Split-CSV $EndIPs

  # Count mismatch check

  for ($i ...) {

  $existing = Get-AzSqlServerFirewallRule ... -ErrorAction SilentlyContinue

  if ($existing) {

  if (same start+end) { [SKIP] } else { Set- (UPDATE) }

  } else { New-AzSqlServerFirewallRule (ADD) }

  }

  }
```

SQL Server block. Uses a start IP + end IP range model (unlike the single-IP model of Storage/KeyVault). Validates that all three lists (rule names, start IPs, end IPs) have equal counts before proceeding. For each rule: if it exists with the same IP range it is skipped; if it exists with different IPs it is updated with Set-AzSqlServerFirewallRule; if it is brand new it is created with New-AzSqlServerFirewallRule. No priority needed for SQL.

Lines 197–291

```
function Set-WebAppIPRules {

param($AppName, $AppLabel, $RuleNamesParam, $IPAddressesParam, $PrioritiesParam)

[string[]]$ruleList = Split-CSV $RuleNamesParam # always array

[string[]]$resolvedIPs = Split-CSV $IPAddressesParam | Resolve-IPEntry

[string[]]$priorityList = Split-CSV $PrioritiesParam # always array

# Build HashSet of all taken priorities

$takenPriorities = [HashSet[int]]::new()

foreach ($r in $existingRules) { $takenPriorities.Add($r.Priority) }

for ($i ...) {

$nameMatch = $existingRules | Where-Object { $_.Name -eq $ruleName }

$ipMatch = $existingRules | Where-Object { $_.IpAddress -eq $ipCidr }

if ($nameMatch) { [SKIP] rule name already exists }

elseif ($ipMatch){ [SKIP] IP already whitelisted }

else {

$finalPri = Resolve-FreePriority $requestedPri $takenPriorities

Add-AzWebAppAccessRestrictionRule ...

$takenPriorities.Add($finalPri)

}

}

}
```

Shared function used by both Function App and App Service blocks. All script-scope variables are passed explicitly as parameters — PowerShell functions do NOT inherit script scope, and the absence of explicit passing was the original bug. The function implements four decision scenarios (detailed in Section 4), builds a live priority HashSet before the loop, and updates it after each successful add so intra-batch rules do not collide with each other.

Lines 298–312

```
if ($runFunction) {

Set-WebAppIPRules -AppName $FunctionAppName -AppLabel 'AZURE FUNCTION'

-RuleNamesParam $RuleNames -IPAddressesParam $IPAddresses

-PrioritiesParam $Priorities

}

if ($runAppService) {

Set-WebAppIPRules -AppName $AppServiceName -AppLabel 'APP SERVICE'

-RuleNamesParam $RuleNames -IPAddressesParam $IPAddresses

-PrioritiesParam $Priorities

}
```

Entry points for Function App and App Service. Both call the same Set-WebAppIPRules function with their respective app names. Both can be enabled in the same pipeline run — they execute sequentially and independently. The same RuleNames, IPAddresses, and Priorities are applied to each app separately.

4. Tested Scenarios — WebApp / Function App

The Set-WebAppIPRules function evaluates each incoming rule against the app's existing IpSecurityRestrictions in the following priority order. The first matching condition wins and the rest are not evaluated.

Scenario 1: Rule name exists — SKIP	
Rule Name Match	YES — rule name found in existing rules
IP Match	Any (not checked)
Priority Match	Any (not checked)
Action	**SKIP with message: 'Rule name already exists'. The rule is not modified or duplicated.**

Scenario 2: IP already whitelisted under different rule name — SKIP	
Rule Name Match	NO — rule name not found
IP Match	YES — same IP/CIDR found under a different rule name
Priority Match	Any (not checked)
Action	**SKIP with message: 'IP already exists under rule X'. Prevents duplicate IP entries.**

Scenario 3: New rule, new IP, priority already taken — AUTO-REASSIGN	
Rule Name Match	NO
IP Match	NO
Priority Match	YES — priority number already used by another rule
Action	**Walk up from requested priority (100→101→102...) until a free slot is found. Log: [PRIORITY CONFLICT] Priority N taken. Auto-assigned: M. Then ADD.**

Scenario 4: New rule, new IP, priority free — ADD	
Rule Name Match	NO
IP Match	NO
Priority Match	NO — priority number is available
Action	**Add rule immediately with the exact requested priority. Log: [ADDED]**

Intra-batch collision prevention: When adding multiple rules in a single pipeline run, the HashSet of taken priorities is updated after each successful add. This means if you submit rules A(p=100), B(p=100), C(p=100) and 100 is already taken, they will land at 101, 102, 103 respectively — not all trying to land on 101.

Priority Resolution Example

Rule	Requested Priority	Taken (at time of processing)	Final Priority
rule-A	100	100 (pre-existing)	101 ← auto-assigned
rule-B	100	100, 101	102 ← auto-assigned
rule-C	105	100, 101, 102	105 ← used as-is
rule-D	102	100, 101, 102, 105	103 ← auto-assigned

5. Storage Account & Key Vault Scenarios

Storage Account and Key Vault use a simpler IP-only model with no rule names or priorities.

Condition	Storage Action	Key Vault Action
IP already in network rules	SKIP — [SKIP] ip already exists	SKIP — normalises /32 suffix before comparing
IP not present	ADD via Add-AzStorageAccountNetworkRule	ADD via Add-AzKeyVaultNetworkRule
Invalid CIDR prefix (outside /24-/31)	EXIT 1 — pipeline fails with clear error	EXIT 1 — pipeline fails with clear error
Invalid IP format	EXIT 1 — pipeline fails with clear error	EXIT 1 — pipeline fails with clear error

6. SQL Server Scenarios

SQL Server uses named firewall rules with a start IP and end IP range.

Condition	Action
Rule name exists AND same start/end IPs	SKIP — [SKIP] Same IPs already set
Rule name exists BUT different start or end IP	UPDATE via Set-AzSqlServerFirewallRule
Rule name does not exist	ADD via New-AzSqlServerFirewallRule
RuleNames / StartIPs / EndIPs count mismatch	EXIT 1 — pipeline fails immediately
Invalid IP format in StartIPs or EndIPs	EXIT 1 — pipeline fails immediately

7. Bugs Fixed During Development

Bug 1 — Character indexing (Processing: s | 1/32 | Priority 49)

Root cause: Split-CSV returned a plain [string] for single-element input. PowerShell's [0] index on a string returns the first character, not the whole string. So 'siva2'[0]='s', '149.5.107.175'[0]='1' (then '/32' appended='1/32'), '111'[0]='1' cast as int=1... but the priority 49 seen in logs was from Azure's default deny-all rule being iterated. Fix: explicit [string[]] cast on every Split-CSV assignment.

Bug 2 — Function App RG returning null

Root cause: Get-AzFunctionApp is not available in all Az module versions on Azure DevOps hosted agents. The function returned null, $rg was empty, and all subsequent Az cmdlets failed. Fix: use Get-AzWebApp for both Function Apps and App Services — they are the same resource type.

Bug 3 — Script-scope variables not visible inside functions

Root cause: PowerShell functions create their own scope. $RuleNames, $IPAddresses, $Priorities were all empty inside Set-WebAppIPRules, causing Split-CSV to return empty arrays, and the loop processed Azure's built-in default rules instead of the user's input. Fix: all script-scope values are passed explicitly as named parameters to the function.

Bug 4 — Same priority accepted for different IPs

Root cause: No duplicate priority check was in place. Azure allows multiple rules with the same priority number but the behaviour is undefined/unpredictable. Fix: Resolve-FreePriority function with a HashSet walks up from the requested number until a free slot is found. The HashSet is updated per-rule within the same batch.

8. Pipeline Log Message Reference

Every outcome produces a distinctly coloured and prefixed log line for easy scanning.

Log Message	Meaning	Colour
`[ADDED] ruleName -> ip | Priority:N`	New rule created successfully	Green
`[UPDATED] ruleName`	SQL rule updated with new IP range	Green
`[SKIP] Rule name 'X' already exists`	Rule name duplicate — no action taken	Yellow
`[SKIP] IP 'X' already exists under rule 'Y'`	IP duplicate under different name — no action taken	Yellow
`[SKIP] Same IPs already set`	SQL rule unchanged — IPs match exactly	Yellow
`[PRIORITY] N is taken, trying N+1...`	Priority conflict detected — walking up	DarkYellow
`[PRIORITY CONFLICT] N taken. Auto-assigned M`	Final auto-assigned priority announced	Yellow
`COUNT MISMATCH DETECTED`	Unequal list lengths — pipeline exits	Red (error)
`RAW inputs — Rules:[...] IPs:[...] ...`	Debug line showing exact received values	White
`Existing priorities in use: 100, 101, ...`	Snapshot of taken priorities before loop starts	White